Capturas de pantalla de los ejercicios nivel 1 y 2 de la tarea 1 del Sprint02

Yonadxandi Manriquez
P2P: Alicia Domínguez

Ejercicio 1
A partir de los documentos adjuntos (estructura_datos y datos_introducir), importa las dos tablas. Muestra las principales características del esquema creado y explica las diferentes tablas y variables que existen. Asegúrate de incluir un diagrama que ilustre la relación entre las diferentes tablas y variables

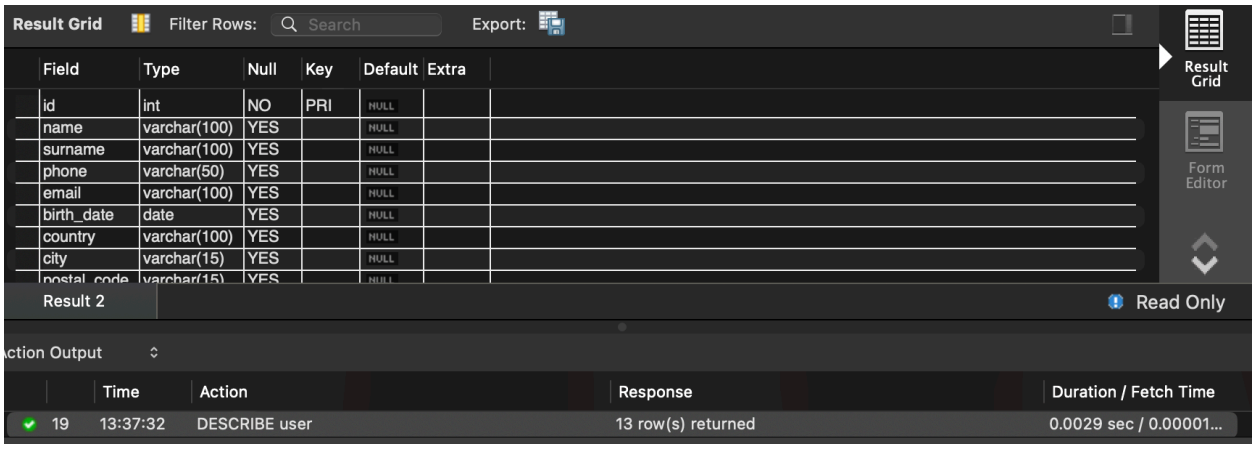
Nota: No puse capturas de pantalla, porque se me olvido, es por ello que colocó las consultas

```
 -- Creamos la tabla company
    CREATE TABLE IF NOT EXISTS company (
        id VARCHAR(15) PRIMARY KEY,
        company_name VARCHAR(255),
        phone VARCHAR(15),
        email VARCHAR(100),
        country VARCHAR(100),
        website VARCHAR(255)
    );

-- Creamos la tabla credit_card

     CREATE TABLE IF NOT EXISTS credit_card(
           id VARCHAR(15) PRIMARY KEY,
           iban VARCHAR(50),
           pan VARCHAR(20),
           pin VARCHAR(10),
           cvv VARCHAR(5),
           expiring_date VARCHAR(10)
     );

    -- Creamos la tabla transaction
--La tabla al crearla, se olvido aplicar las buenas prácticas, colocar los contrains al final, es por ello  que se borró y se volvio a crear

  DROP TABLE IF EXISTS transaction;
```

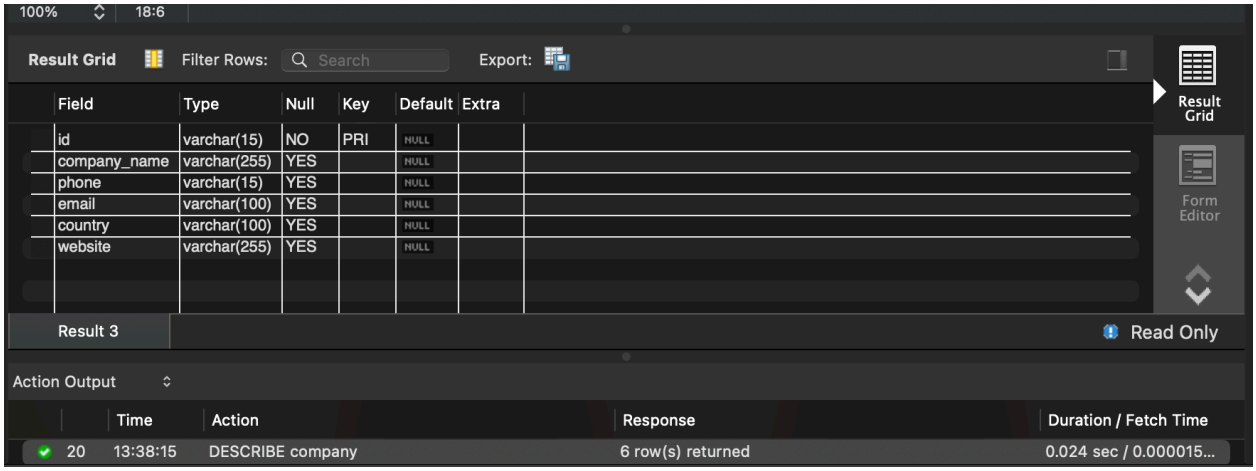
```
CREATE TABLE IF NOT EXISTS transaction (
    id VARCHAR(255),
    credit_card_id VARCHAR(15),
    company_id VARCHAR(20),
    user_id INT,
    lat FLOAT,
    longitude FLOAT,
    timestamp TIMESTAMP,
    amount DECIMAL(10, 2),
    declined BOOLEAN,

    CONSTRAINT pk_transaction PRIMARY KEY (id),
    CONSTRAINT fk_transaction_credit_card FOREIGN KEY (credit_card_id)
REFERENCES credit_card(id),
    CONSTRAINT fk_transaction_user FOREIGN KEY (user_id) REFERENCES
user(id),
    CONSTRAINT fk_transaction_company FOREIGN KEY (company_id)
REFERENCES company(id)
);
```

2. Mostrar las características generales del esquema para ello usaremos la expresión

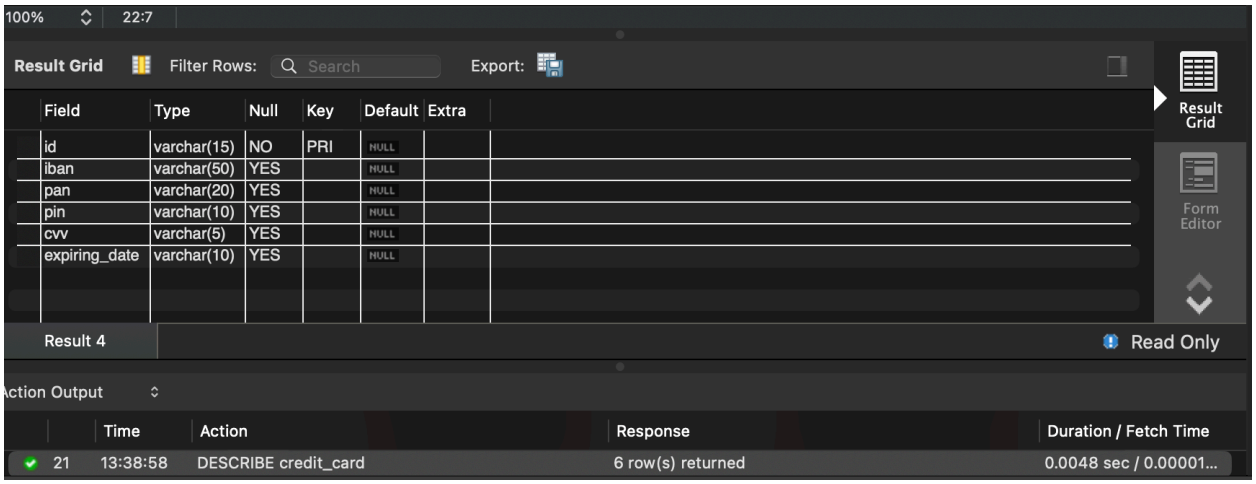
```
DESCRIBE user;
```



```
DESCRIBE company;
```

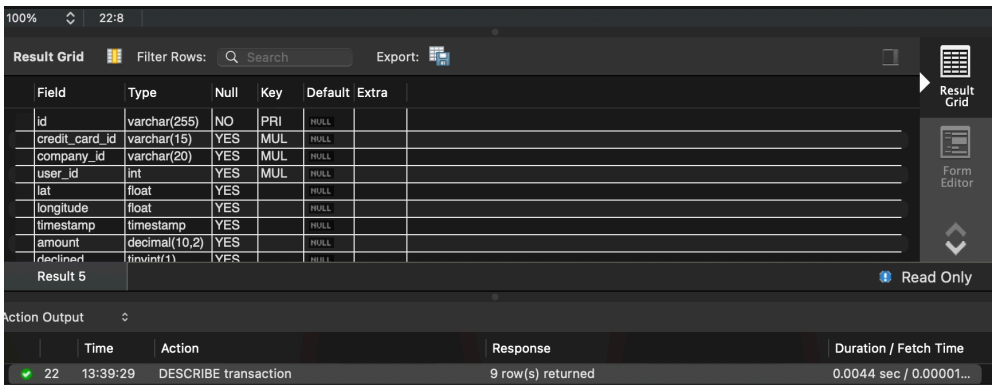


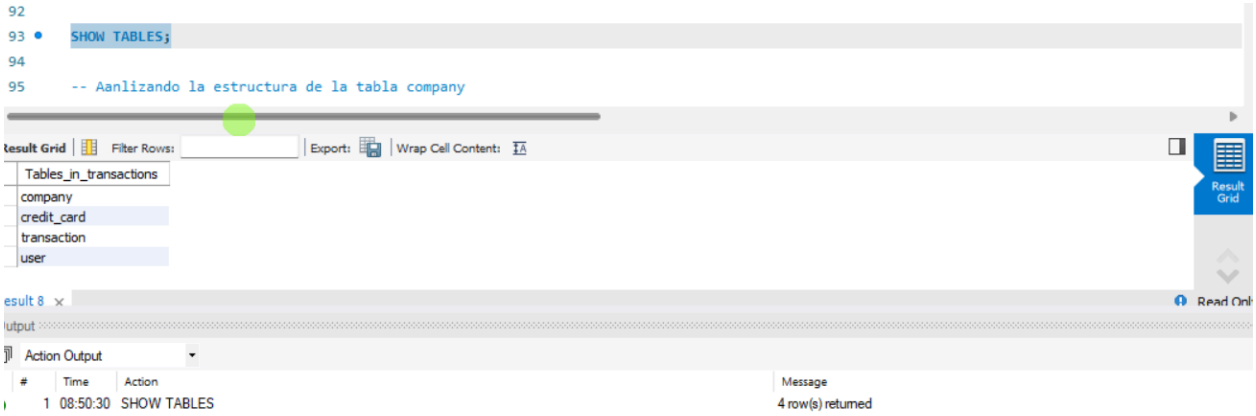

```
DESCRIBE credit_card;
```



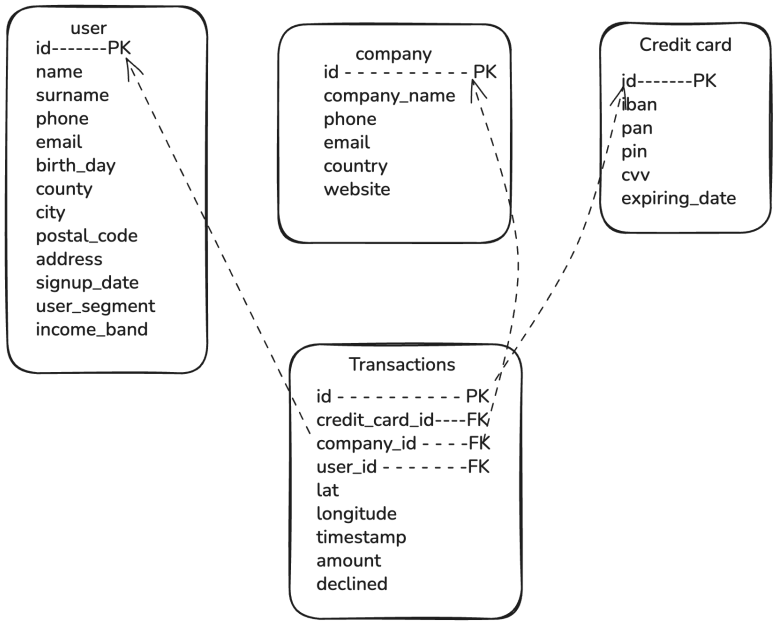
Las columnas KEY podemos observar los PK, principalmente de user, company y credit_card por lo tanto son tablas padres. Para que podamos aplicar la PK, se necesita que esta característica no reciba valores null, el resto al no indicarlo explícitamente, los admite.

```
DESCRIBE transaction;
```





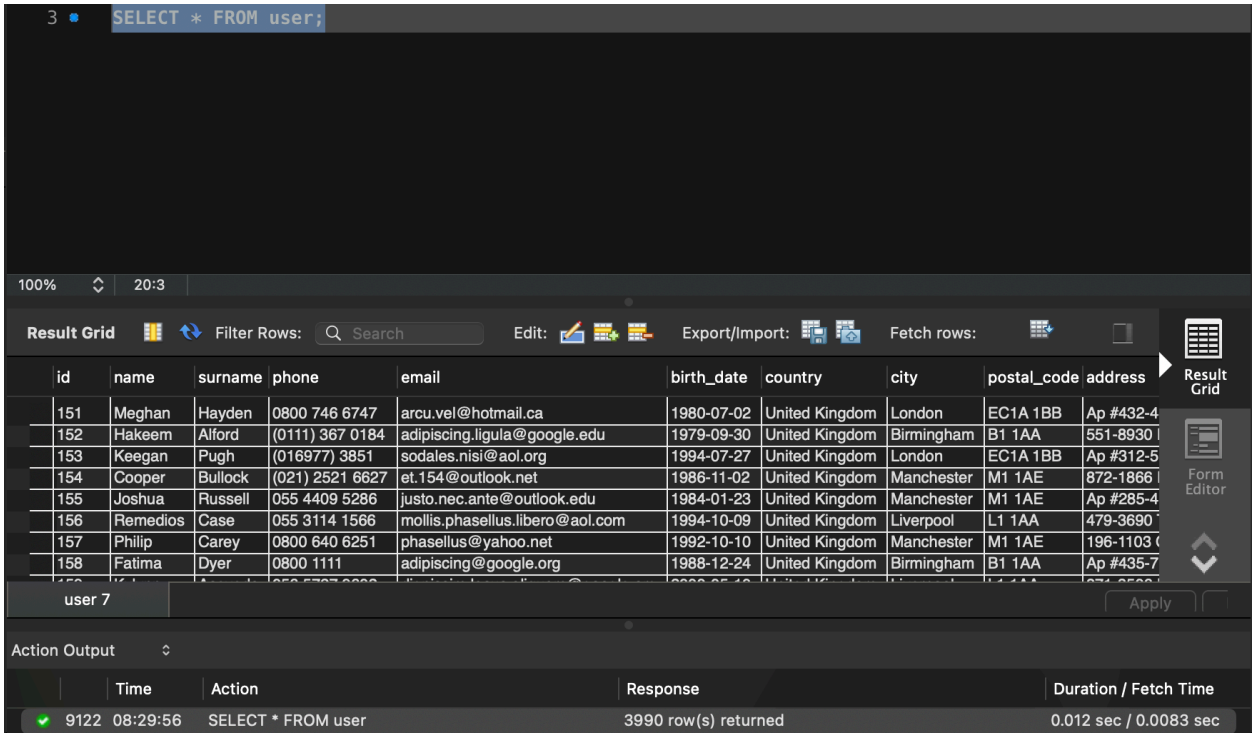
En esta tabla (tabla hijo) podemos ver que además del PK tiene sus enlaces con las otras tablas a través de los FK que en este caso se representan como MUL. Esta tabla de transacciones podemos observar varios tipos, y uno de ellos que no se observa en las tablas anteriores es



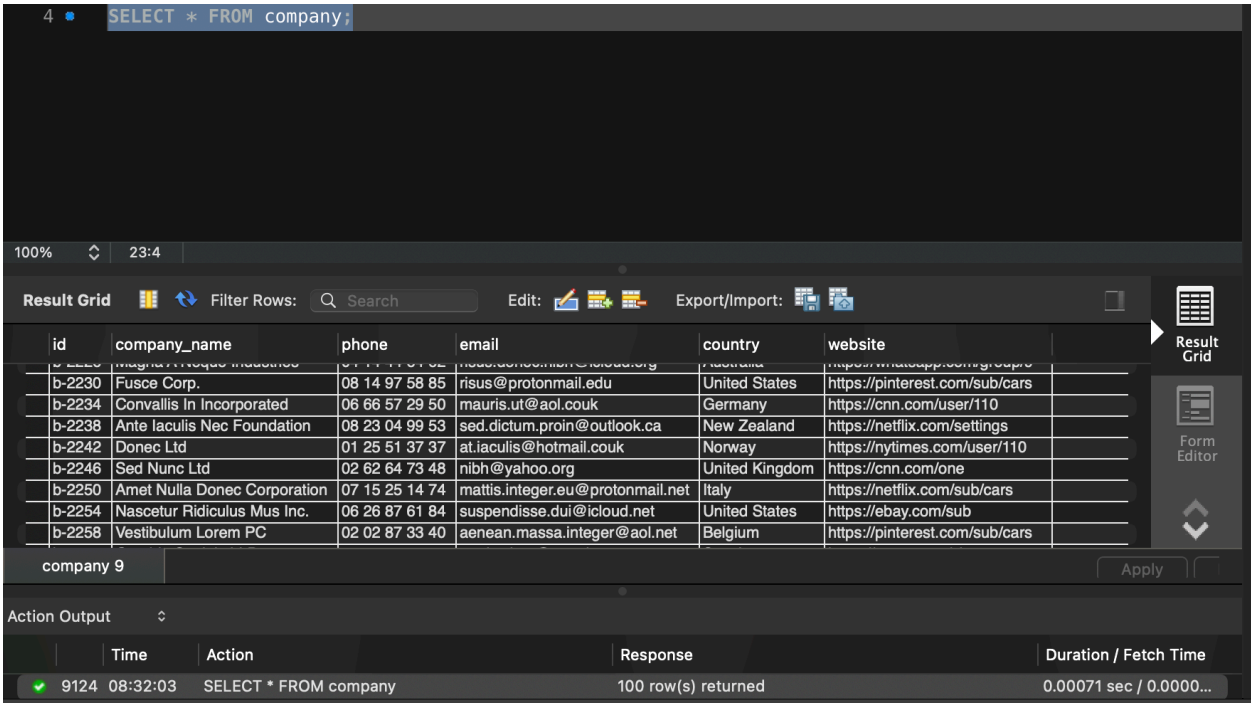
3. Diagrama de relación entre tablas

INSERT VALUES de las tablas creadas

user



company



credit_card

```
5  SELECT * FROM credit_card;
6
```

Result Grid | Filter Rows: Search | Edit: | Export/Import: | Fetch rows:

id	iban	pan	pin	cvv	expiring_date
CcS-4857	XX485759183529250585077 1	2314242385113924	1819	467	09/27/25
CcS-4858	XX858176813700243609402 5	6582720299715533	3964	817	12/28/28
CcS-4859	XX782693049142355360937 0	8861684536289642	4983	277	11/26/26
CcS-4860	XX555959036883530464529 9	2481155515498459	6876	661	07/27/27
CcS-4861	XX203518287719519162730 7	1308930301149557	5710	398	04/25/26
CcS-4862	XX477472146246364540975 8	6715617009807829	4042	174	11/27/26
CcS-4863	XX147682966424504620711 1	3140879819451394	5969	449	12/27/29
CcS-4864	XX838029889338573119615 9	5793672133649114	8481	139	02/28/26

credit_card 10 | Apply

Action Output

Time	Action	Response	Duration / Fetch Time
9125 08:32:53	SELECT * FROM credit_card	5000 row(s) returned	0.0013 sec / 0.0066...

transaction

```
6  SELECT * FROM transaction;
7
8
```

Result Grid | Filter Rows: Search | Edit: | Export/Import: | Fetch rows:

id	credit_card...	company_id	user_id	lat	longitude	timestamp	amount	declined
00045D6B-ED2E-4F2F-8186-CEE074D875D0	CcS-6699	b-2390	2118	29.7573	-95.3796	2020-07-14 15:37:45	326.01	0
000481C3-1C26-4FEF-83A0-4CD0EB004BBD	CcS-6696	b-2230	2115	53.5489	-113.503	2017-09-04 19:44:53	161.60	0
00051AA4-9CBE-4268-B070-C38062A1B3E2	CcS-7606	b-2266	3025	52.2084	5.69081	2017-01-05 18:19:25	148.91	0
0008A312-EDFE-4A4F-BC99-E9C92EC3CA4D	CcU-3358	b-2598	215	53.5535	-113.499	2023-09-23 04:51:43	294.59	0
0009A151-9BCF-4E31-9053-A468FF77FAAB	CcS-7509	b-2546	2928	51.9362	5.34265	2023-12-31 00:06:36	383.63	0
0009D494-6245-4DF9-955D-2C084191CFFB	CcS-8483	b-2526	3902	45.492	-73.5706	2017-07-18 07:52:02	197.80	0
000A1DEC-CDB6-4AB2-A619-71DAB8D4A262	CcS-6467	b-2558	1886	55.7425	-3.30009	2018-09-08 05:29:58	339.94	0
000A1E64-1414-40B0-9D92-5678A4D958E2	CcS-5966	b-2550	1385	52.0821	5.28424	2022-09-17 04:02:19	369.71	0

transaction 19 | Apply

Action Output

Time	Action	Response	Duration / Fetch Time
10... 08:47:43	SELECT * FROM transaction	100000 row(s) returned	0.0015 sec / 0.093 sec

/*__Ejercicio 2__
Utilizando JOIN realizarás las siguientes consultas:

2.1 Listado de los países que están generando ventas.

```
106
107 •  SELECT DISTINCT c.country AS paises_operadores
108    FROM company c
109    JOIN transaction t
110    ON t.company_id = c.id
111    WHERE declined = 0;
112
113    -- Desde cuántos países se generen las ventas.
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

paises_operadores
Netherlands
Sweden
Ireland
United States
Belgium

Result 9

Output

Action Output

#	Time	Action	Message
1	08:53:33	SELECT DISTINCT c.country AS paises_operadores FROM company c JOIN transaction t ON t.company_id = c...	15 row(s) returned

2.2 Desde cuántos países se generen las ventas.

```
112
113    -- 2.2 Desde cuántos países se generen las ventas.
114 •  SELECT COUNT(DISTINCT c.country) AS TotL_paises_operan
115    FROM company c
116    JOIN transaction t
117    ON t.company_id = c.id
118    WHERE declined = 0;
119
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

TotL_paises_operan
15

Result 10

Output

Action Output

#	Time	Action	Message
1	08:53:33	SELECT DISTINCT c.country AS paises_operadores FROM company c JOIN transaction t ON t.company_id = c...	15 row(s) returned
2	08:54:44	SELECT COUNT(DISTINCT c.country) AS TotL_paises_operan FROM company c JOIN transaction t ON t.com...	1 row(s) returned

-- 2.3 Compañia con mayor media de ventas

```sql
-- 2.3  Compañia con mayor media de ventas

SELECT c.company_name,
ROUND(AVG(t.amount),3) AS media_ventas
FROM company c
JOIN transaction t
ON t.company_id = c.id
WHERE declined = 0
GROUP BY c.company_name, c.id
ORDER BY AVG(t.amount) DESC
LIMIT 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

company_name	media_ventas
Ac Fermentum Incorporated	284.911

Result 11

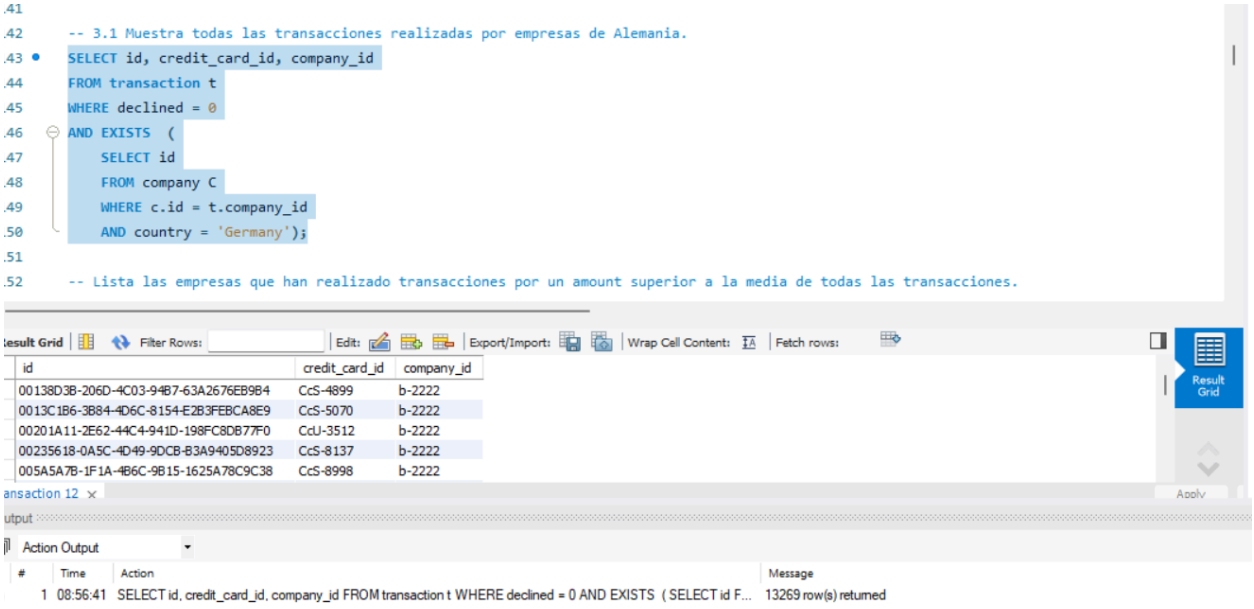
Output

Action Output

#	Time	Action	Message
1	08:55:31	SELECT c.company_name, ROUND(AVG(t.amount),3) AS media_ventas FROM company c JOIN transaction t ...	1 row(s) returned

/***Ejercicio 3.** Utilizando sólo subconsultas (sin utilizar JOIN):

Muestra todas las transacciones realizadas por empresas de Alemania.
Lista las empresas que han realizado transacciones por un amount superior a
la media de todas las transacciones.
Eliminarán del sistema las empresas que carecen de transacciones registradas, entrega el listado de estas empresas.*/



```
41
42    -- 3.1 Muestra todas las transacciones realizadas por empresas de Alemania.
43    SELECT id, credit_card_id, company_id
44    FROM transaction t
45    WHERE declined = 0
46    AND EXISTS  (
47        SELECT id
48        FROM company C
49        WHERE c.id = t.company_id
50        AND country = 'Germany');
51
52    -- Lista las empresas que han realizado transacciones por un amount superior a la media de todas las transacciones.
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

id	credit_card_id	company_id
00138D3B-206D-4C03-94B7-63A2676EB9B4	CcS-4899	b-2222
0013C1B6-3B84-4D6C-8154-E2B3FEBCA8E9	CcS-5070	b-2222
00201A11-2E62-44C4-941D-198FC8DB77F0	CcU-3512	b-2222
00235618-0A5C-4D49-9DCB-B3A9405D8923	CcS-8137	b-2222
005A5A7B-1F1A-4B6C-9B15-1625A78C9C38	CcS-8998	b-2222

Result Grid

ansaction 12 ×

Apply

utput

Action Output

#	Time	Action	Message
1	08:56:41	SELECT id, credit_card_id, company_id FROM transaction t WHERE declined = 0 AND EXISTS (SELECT id F...	13269 row(s) returned

3.2 Lista las empresas que han realizado transacciones por un amount superior a la media de todas las transacciones.

```sql
-- 3.2 Lista las empresas que han realizado transacciones por un amount superior a la media de todas las transacci

SELECT DISTINCT company_name AS nombre_empresa
FROM company c
WHERE EXISTS (
    SELECT company_id
    FROM transaction t
    WHERE t.company_id = c.id
    GROUP BY company_id
    HAVING AVG(amount) > (SELECT AVG(amount) FROM transaction
    WHERE declined = 0)
);

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

nombre_empresa
Ac Fermentum Incorporated
Magna A Neque Industries
Fusce Corp.
Ante Iaculis Nec Foundation
Donec Ltd

company 13

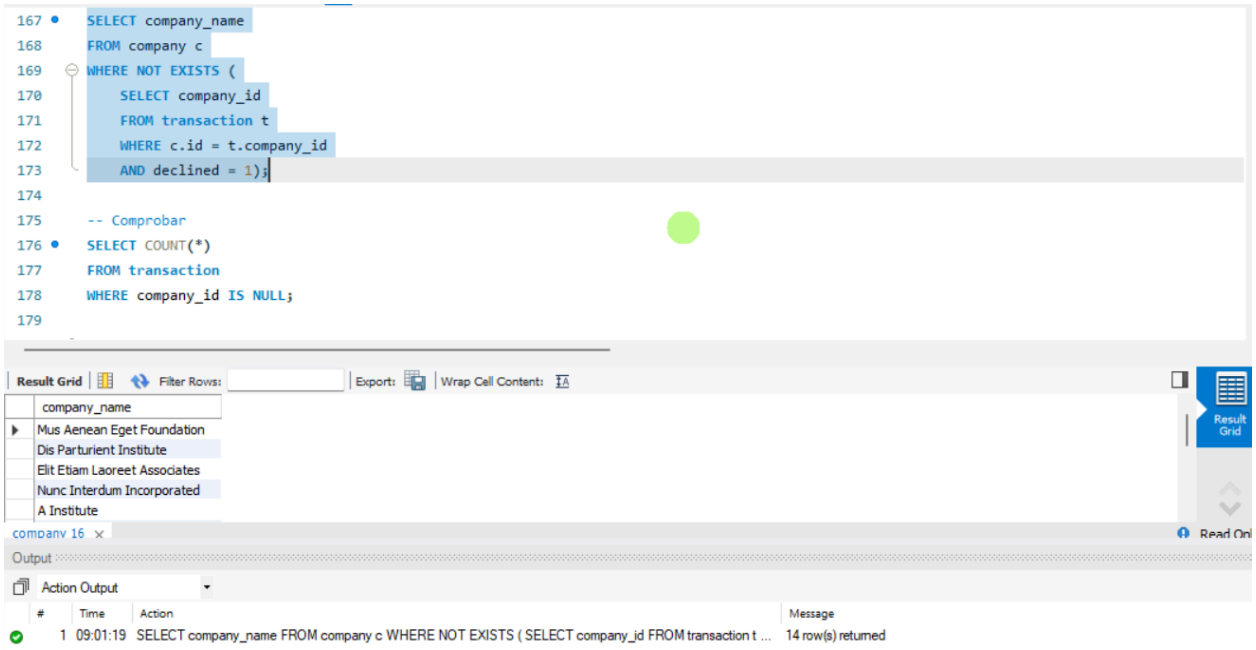
Output

Action Output

#	Time	Action	Message
1	08:58:44	SELECT DISTINCT company_name AS nombre_empresa FROM company c WHERE EXISTS (SELECT comp...	47 row(s) returned

-- 3.3 Eliminarán del sistema las empresas que carecen de transacciones registradas, entrega el listado de estas empresas.

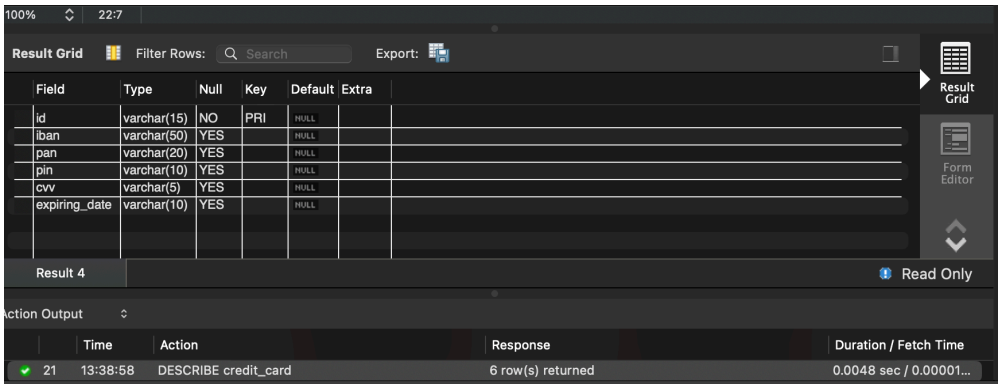
Todas tienen transacciones



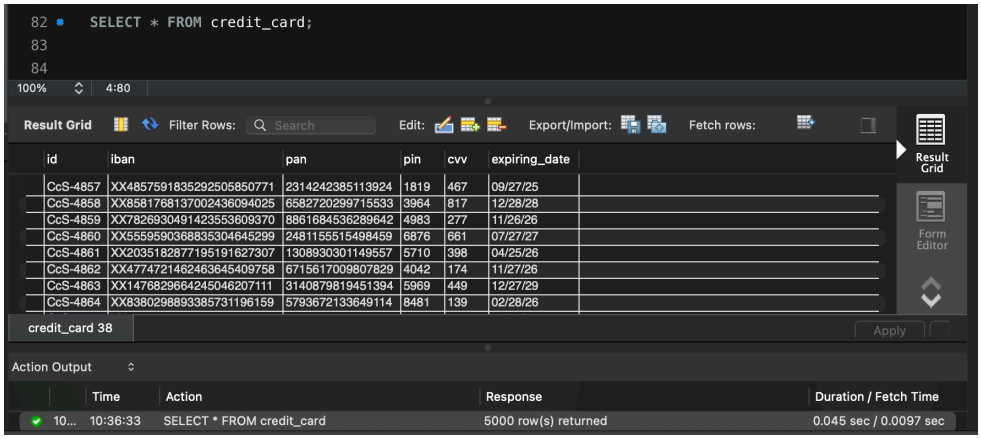
/***Ejercicio 4**
Tu tarea es diseñar y crear una tabla llamada "credit_card" que almacene detalles cruciales sobre las tarjetas de crédito. La nueva tabla debe ser capaz de identificar de forma única cada tarjeta y establecer una relación adecuada con las otras dos tablas ("transaction" y "company"). Después de crear la tabla será necesario que ingreses la información del documento denominado "datos_introducir_credit".

Recuerda mostrar el diagrama y realizar una breve descripción del mismo.*/

Nota: Esta tabla se creo desde el ejercicio 1, para que entendiese mejor como estaban las relaciones.



Introducir datos se hizo en el ejercicio _01 ejecutando el fichero de N1-Ex.4__ datos_introducir_credit.sql



Revisión

```
218
219     -- Revisión de la estructura de tablas
220
221 •   SHOW CREATE TABLE transaction;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Table	Create Table
transaction	CREATE TABLE `transaction` (`id` varchar(255) NOT NULL, `credit_card_id` varchar(15) DEFAULT NULL, `company_id` varchar(20) DEFAULT NULL, `user_id` int DEFAUL...

-- Cambio de VRACHAR A DATE
Cambio que me recomendaron en el P2P para que las fechas estuvieran en una estructura correcta para leer

```
211 •   UPDATE credit_card
212     SET expiring_date = STR_TO_DATE(expiring_date, '%m/%d/%y')
213     WHERE id IS NOT NULL
214     LIMIT 9999999999;
215
216     -- Comprobación de datos una vez hecho la conversion
217 •   SELECT * FROM credit_card
218     LIMIT 100;
219
220 •   SELECT COUNT(*)
221     FROM credit_card
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	iban	pin	cvv	expiring_date
CcS-4857	XX485759183529250585077 1	1819	467	2025-09-27
CcS-4858	XX858176813700243609402 5	3964	817	2028-12-28
CcS-4859	XX782693049142355360937 0	4983	277	2026-11-26
CcS-4860	XX555959036883530464529 9	6876	661	2027-07-27
CcS-4861	XX203518287719519162730 7	5710	398	2026-04-25

credit_card 21 ×

Output

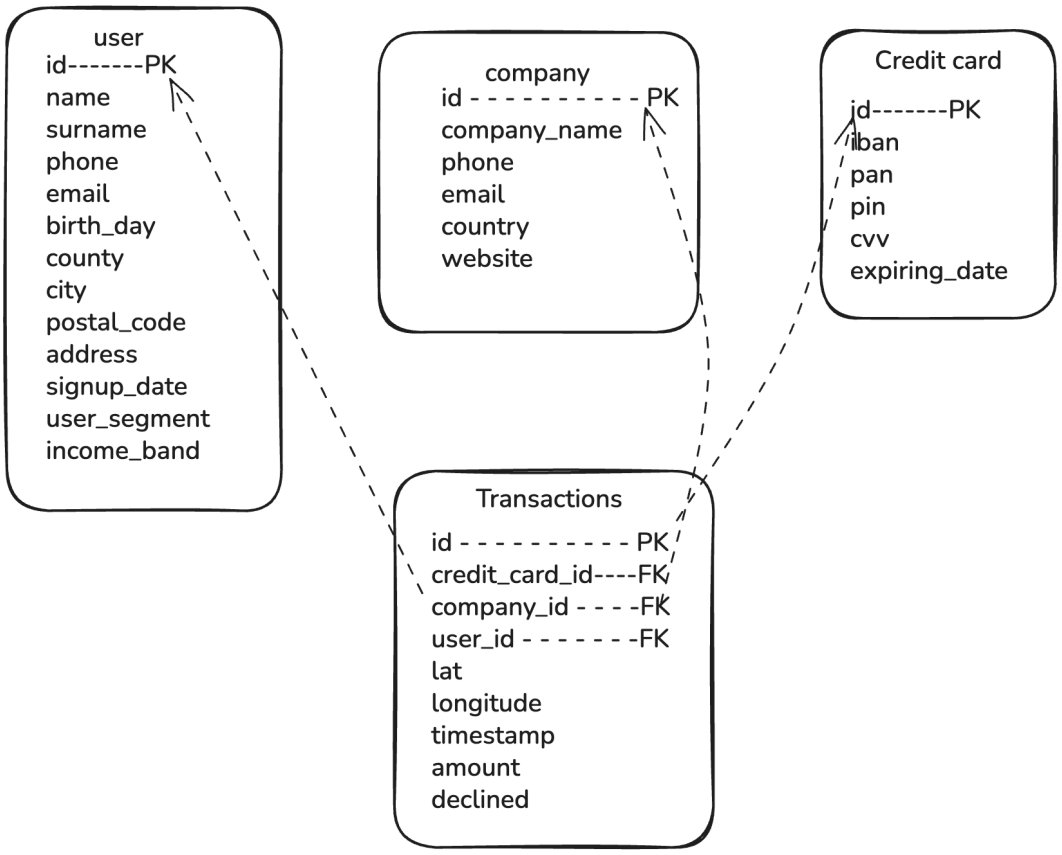
Action Output

#	Time	Action	Message
1	09:05:49	SELECT * FROM credit_card LIMIT 100	100 row(s) returned
2	09:05:59	SELECT COUNT(*) FROM credit_card WHERE expiring_date IS NULL	1 row(s) returned
3	09:06:22	SELECT * FROM credit_card LIMIT 100	100 row(s) returned

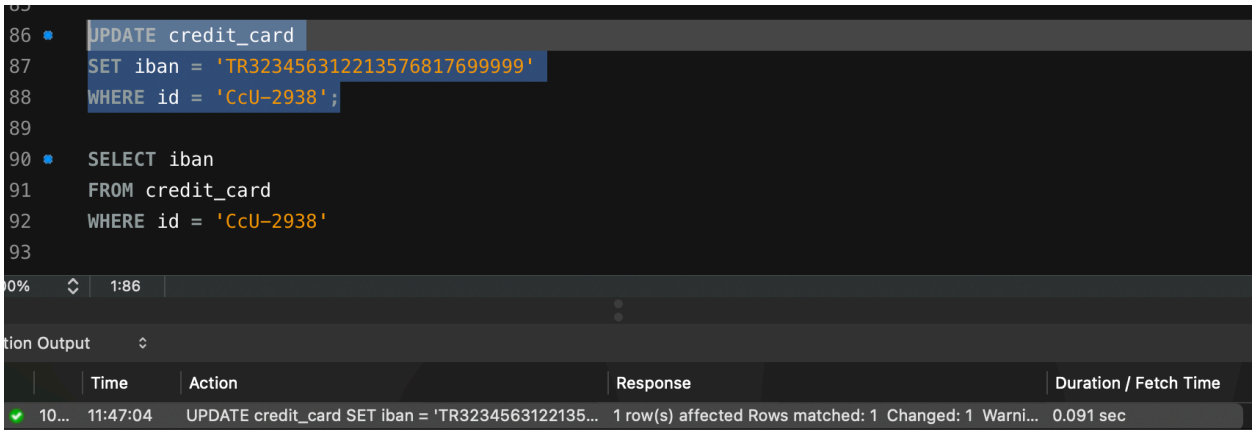
-- Cambio del tipo de columna a DATE
ALTER TABLE credit_card
MODIFY expiring_date DATE;

Diagrama de relación

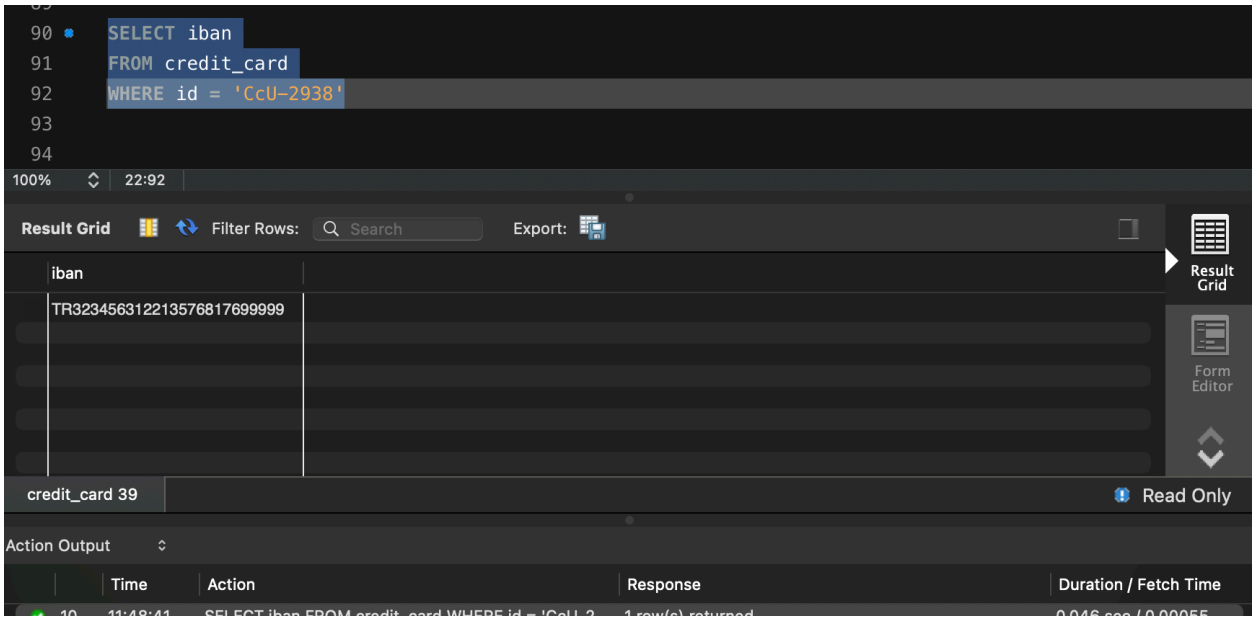
La base de datos transaction tiene 4 tablas donde la tabla hija es transaction que se relaciona con las tablas padre a través de sus claves foráneas. Esta estructura permite consultar información cruzada, como qué usuario realizó una transacción, en qué empresa y con qué tarjeta.



/*Ejercicio 5
El departamento de Recursos Humanos ha identificado un error en el número de cuenta asociado a su tarjeta de crédito con ID CcU-2938. La información que debe mostrarse para este registro es: TR323456312213576817699999. Recuerda mostrar que el cambio se realizó*/



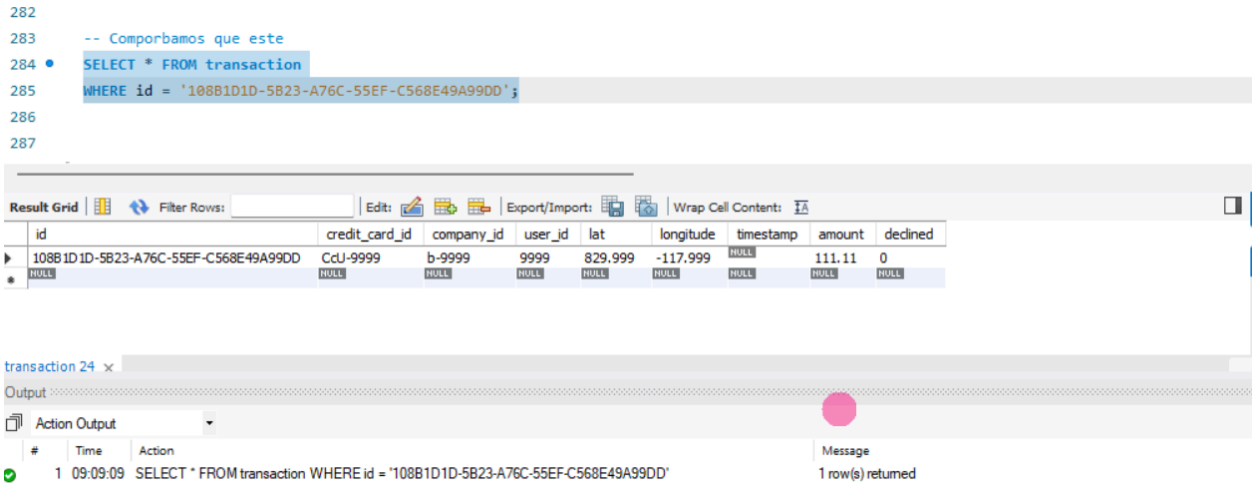
Comprobación



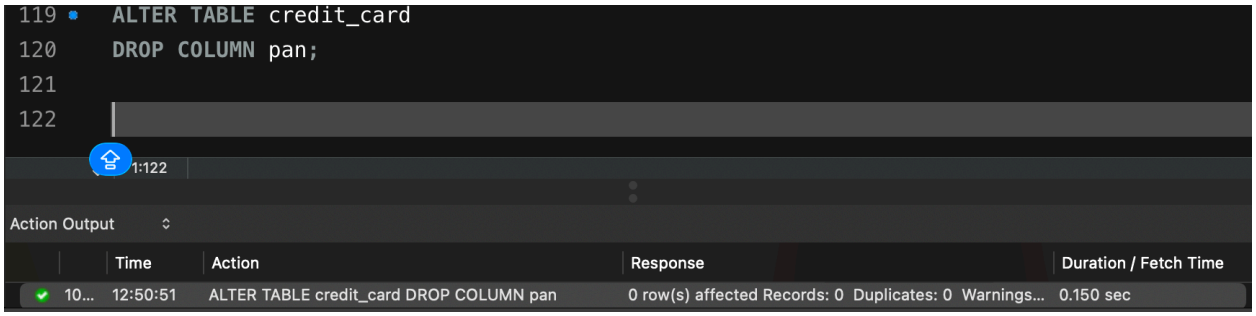
/*__Ejercicio 6__
En la tabla "transaction" ingresa una nueva transacción con la siguiente información*/

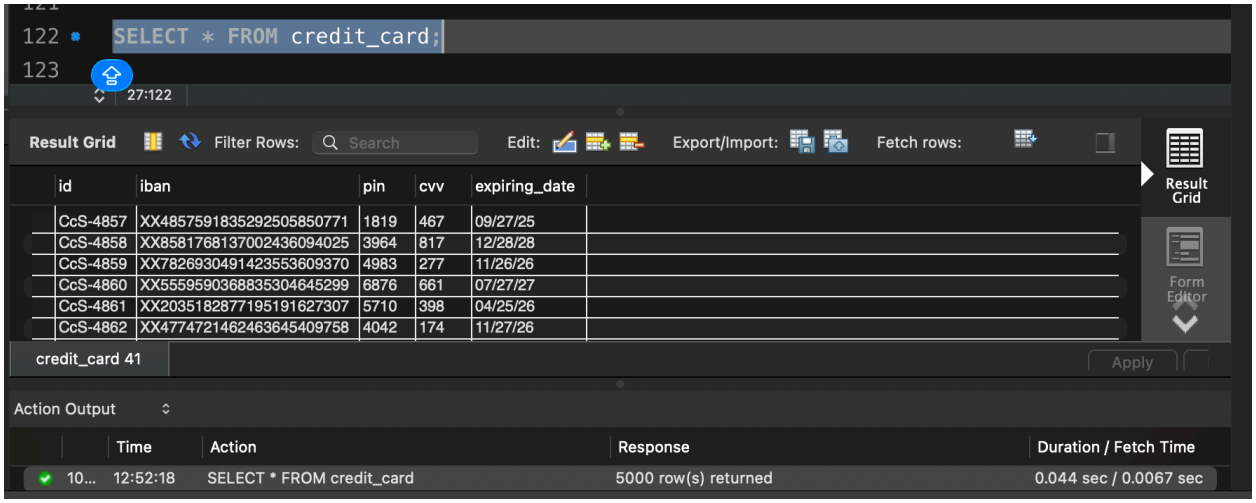
Una cosa importante, se revisaron los datos tanto de los archivos de insert como las tablas de las tablas correspondientes y los valores que se piden introducir no existen en las tablas padre y nos daba error, al no poder relacionar.

Para respetar la integridad referencial, los insertamos primero en sus tablas padre antes de insertarlos en transaction



/*__Ejercicio 7__
Desde recursos humanos te solicitan eliminar la columna "pan" de la tabla credit_card.
Recuerda mostrar el cambio realizado.*/

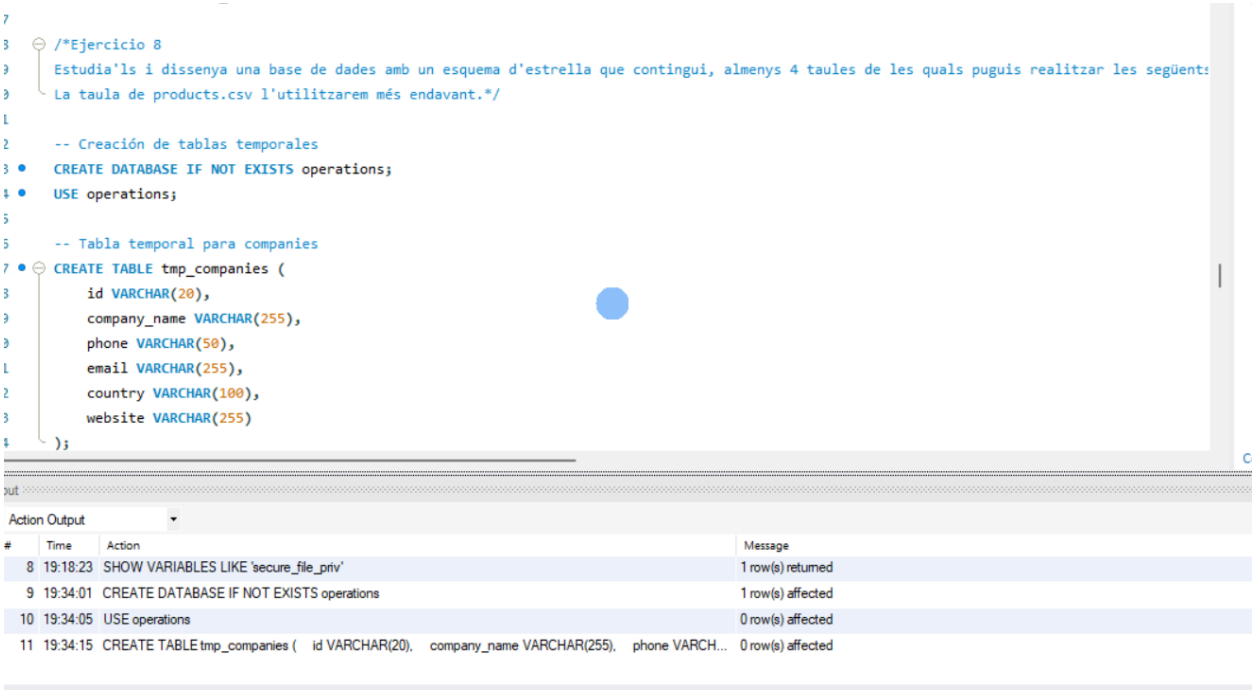




/*__Ejercicio 8__
Estudia y diseña una base de datos con un esquema de estrella que contenga, al menos 4 tablas de los que puedas realizar las siguientes consultas:
La tabla de products.csv la utilizaremos más adelante.*/

-8.1 En el P2P me recomendaron hacer una base de datos para colocar las nuevas tablas, ya que no tienen la misma longitud que las tablas de los ejercicios anteriores, es por ello que creamos la base de datos llamada **operations.**

-- 8.2 Creación de tablas temporales para poder vaciar los datos mas fácil como los de fecha y decimales
-- ponemos todo como VARCHAR para que no haya errores desde el inicio en MySQL



```sql
/*Ejercicio 8
Estudia'ls i dissenya una base de dades amb un esquema d'estrella que contingui, almenys 4 taules de les quals puguis realitzar les següents
La taula de products.csv l'utilitzarem més endavant.*/

-- Creación de tablas temporales
CREATE DATABASE IF NOT EXISTS operations;
USE operations;

-- Tabla temporal para companies
CREATE TABLE tmp_companies (
    id VARCHAR(20),
    company_name VARCHAR(255),
    phone VARCHAR(50),
    email VARCHAR(255),
    country VARCHAR(100),
    website VARCHAR(255)
);
```

Action Output

#	Time	Action	Message
8	19:18:23	SHOW VARIABLES LIKE 'secure_file_priv'	1 row(s) returned
9	19:34:01	CREATE DATABASE IF NOT EXISTS operations	1 row(s) affected
10	19:34:05	USE operations	0 row(s) affected
11	19:34:15	CREATE TABLE tmp_companies (id VARCHAR(20), company_name VARCHAR(255), phone VARCH...	0 row(s) affected

```sql
-- Tabla temporal para credit_cards
CREATE TABLE tmp_credit_cards (
    id VARCHAR(20),
    user_id VARCHAR(20),
    iban VARCHAR(50),
    pan VARCHAR(50),
    pin VARCHAR(10),
    cvv VARCHAR(10),
    track1 VARCHAR(255),
    track2 VARCHAR(255),
    expiring_date VARCHAR(20)
);

-- Tabla temporal para transactions
CREATE TABLE tmp_transactions (
    id VARCHAR(20),
```

Output

Action Output

#	Time	Action	Message
1	19:36:21	CREATE TABLE tmp_american_users (id VARCHAR(20), name VARCHAR(255), surname VARCHAR(25...	Error Code: 1050. Table 'tmp_american_users' already exists
2	19:36:30	CREATE TABLE tmp_european_users (id VARCHAR(20), name VARCHAR(255), surname VARCHAR(25...	0 row(s) affected

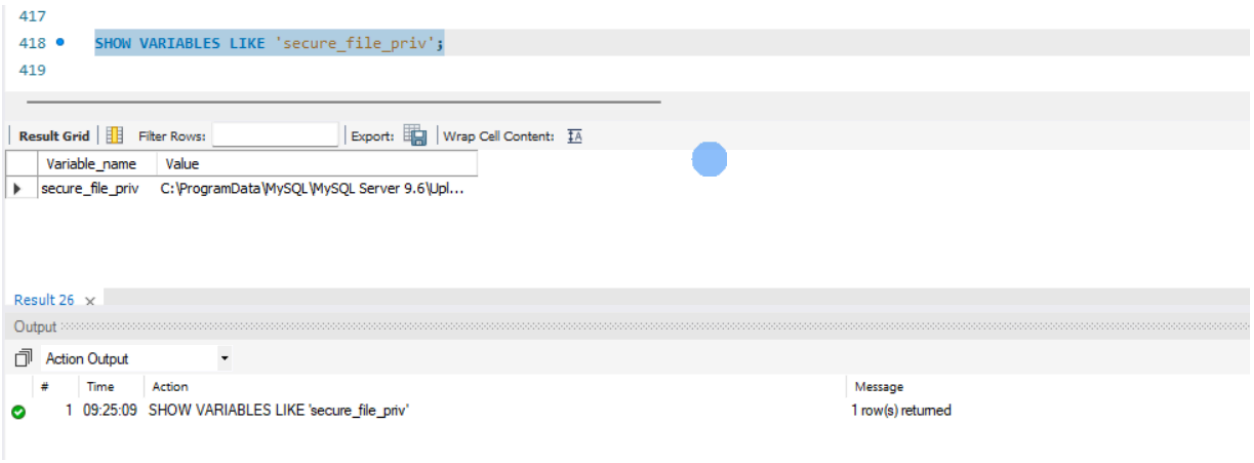
```sql
CREATE TABLE tmp_transactions (
    id VARCHAR(50),
    card_id VARCHAR(20),
    business_id VARCHAR(20),
    timestamp VARCHAR(30),
    amount VARCHAR(20),
    declined VARCHAR(10),
    product_ids VARCHAR(255),
    user_id VARCHAR(20),
    lat VARCHAR(30),
    longitude VARCHAR(30),
    discount_amount VARCHAR(20),
    tax_amount VARCHAR(20),
    shipping_amount VARCHAR(20),
    channel VARCHAR(20),
    campaign_id VARCHAR(50),
    device_type VARCHAR(20),
    is_international VARCHAR(5),
    decline_reason VARCHAR(100),
    distance_km VARCHAR(20)
);
LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 9.6/Uploads/N1-Ex.8  companies.csv'
```

Output

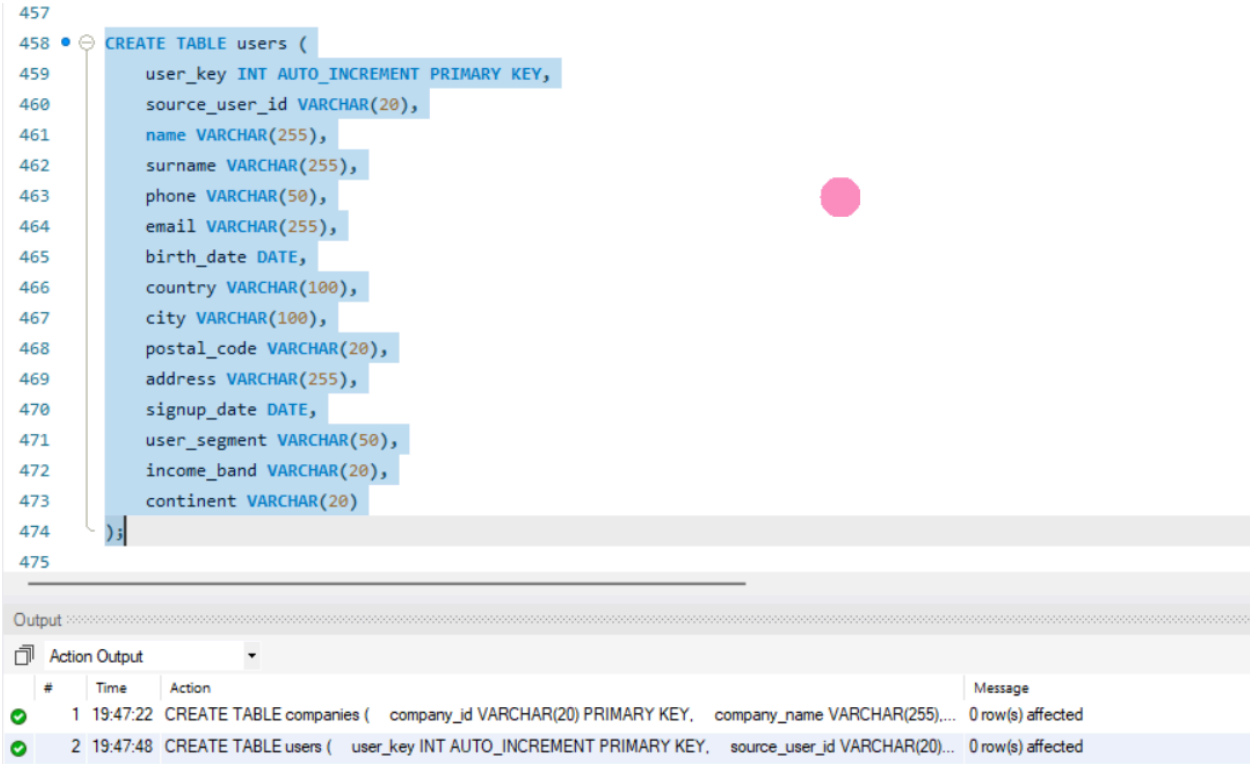
Action Output

#	Time	Action	Message
16	19:42:33	CREATE TABLE tmp_american_users (id VARCHAR(20), name VARCHAR(255), surname VARCHAR(2...	0 row(s) affected
17	19:42:41	CREATE TABLE tmp_european_users (id VARCHAR(20), name VARCHAR(255), surname VARCHAR(...	0 row(s) affected
18	19:42:47	CREATE TABLE tmp_credit_cards (id VARCHAR(20), user_id VARCHAR(20), iban VARCHAR(50), p...	0 row(s) affected
19	19:42:53	CREATE TABLE tmp_transactions (id VARCHAR(50), card_id VARCHAR(20), business_id VARCHAR(2...	0 row(s) affected

```
-- 8.3 Cargar los datos desde los archivos del ordenador
-- Importante: Cambiar esta ruta por la carpeta Uploads de MySQL
-- Ejecutar primero: SHOW VARIABLES LIKE 'secure_file_priv';
```




```
-- 8.4 Crear las Tablas definitivas
```



```sql
445     -- Tablas definitivas
446
447 CREATE TABLE companies (
448     company_id VARCHAR(20) PRIMARY KEY,
449     company_name VARCHAR(255),
450     phone VARCHAR(50),
451     email VARCHAR(255),
452     country VARCHAR(100),
453     website VARCHAR(255),
454     merchant_category VARCHAR(100),
455     merchant_price_position VARCHAR(50)
456 );
457
458 CREATE TABLE users (
459     user_key INT AUTO_INCREMENT PRIMARY KEY,
460     source user id VARCHAR(20),
```

Output

Action Output

#	Time	Action	Message
1	19:47:22	CREATE TABLE companies (company_id VARCHAR(20) PRIMARY KEY, company_name VARCHAR(255),...	0 row(s) affected

```sql
475
476 CREATE TABLE credit_cards (
477     id VARCHAR(20) PRIMARY KEY,
478     user_id INT,
479     iban VARCHAR(50),
480     pan VARCHAR(50),
481     pin VARCHAR(10),
482     cvv VARCHAR(10),
483     track1 VARCHAR(255),
484     track2 VARCHAR(255),
485     expiring_date DATE,
486     card_type VARCHAR(20),
487     card_renewal_flag TINYINT
488 );
489
490 CREATE TABLE transactions (
491     id VARCHAR(100) PRIMARY KEY,
492     card_id VARCHAR(20),
493     business id VARCHAR(20),
```

Output

Action Output

#	Time	Action	Message
1	19:47:22	CREATE TABLE companies (company_id VARCHAR(20) PRIMARY KEY, company_name VARCHAR(255),...	0 row(s) affected
2	19:47:48	CREATE TABLE users (user_key INT AUTO_INCREMENT PRIMARY KEY, source_user_id VARCHAR(20)...	0 row(s) affected
3	19:48:13	CREATE TABLE credit_cards (id VARCHAR(20) PRIMARY KEY, user_id INT, iban VARCHAR(50), pa...	0 row(s) affected

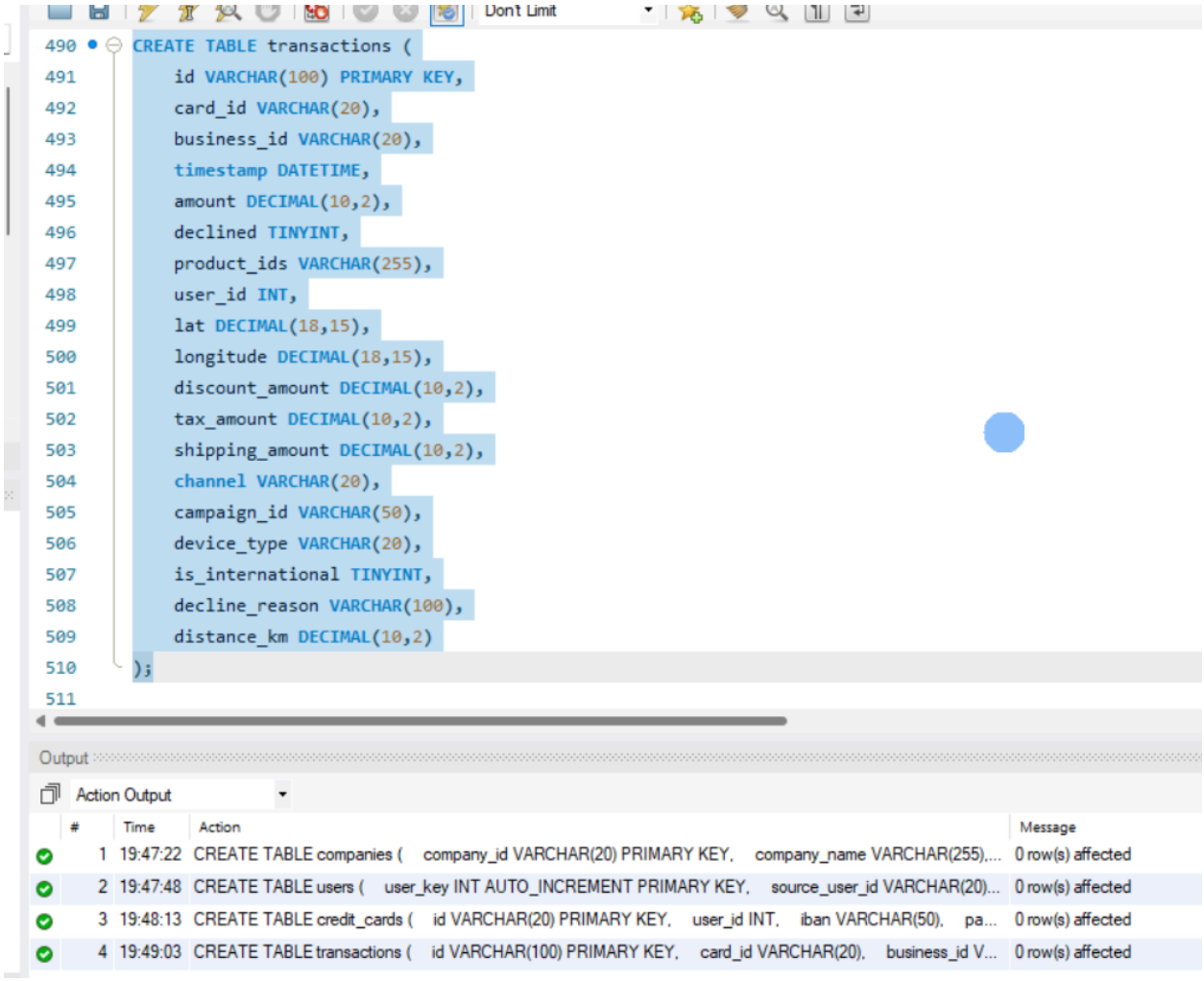
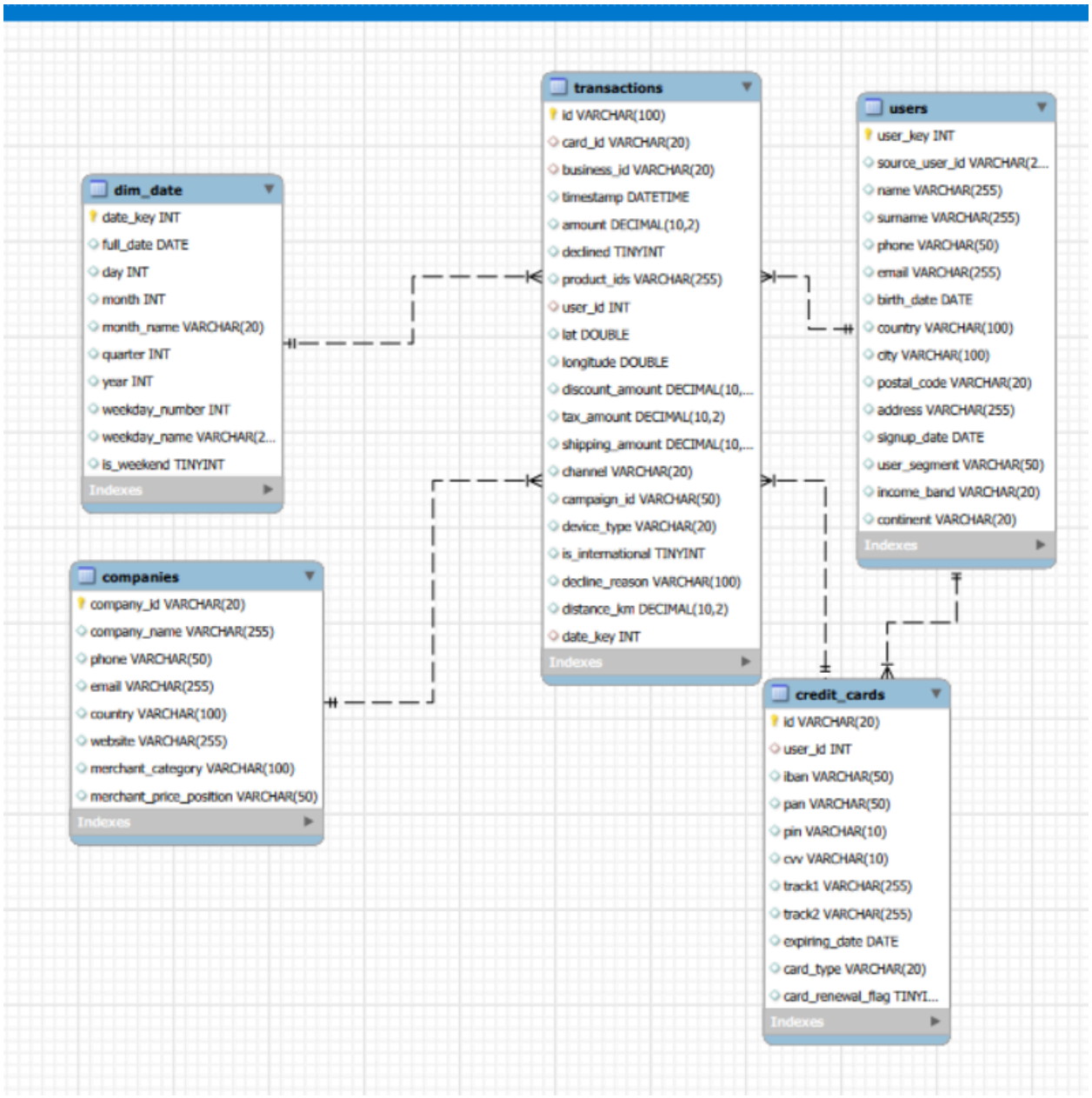

Don't Limit
490 CREATE TABLE transactions (
491 id VARCHAR(100) PRIMARY KEY,
492 card_id VARCHAR(20),
493 business_id VARCHAR(20),
494 timestamp DATETIME,
495 amount DECIMAL(10,2),
496 declined TINYINT,
497 product_ids VARCHAR(255),
498 user_id INT,
499 lat DECIMAL(18,15),
500 longitude DECIMAL(18,15),
501 discount_amount DECIMAL(10,2),
502 tax_amount DECIMAL(10,2),
503 shipping_amount DECIMAL(10,2),
504 channel VARCHAR(20),
505 campaign_id VARCHAR(50),
506 device_type VARCHAR(20),
507 is_international TINYINT,
508 decline_reason VARCHAR(100),
509 distance_km DECIMAL(10,2)
510);
511
Output
Action Output
Time Action Message
1 19:47:22 CREATE TABLE companies (company_id VARCHAR(20) PRIMARY KEY, company_name VARCHAR(255),... 0 row(s) affected
2 19:47:48 CREATE TABLE users (user_key INT AUTO_INCREMENT PRIMARY KEY, source_user_id VARCHAR(20)... 0 row(s) affected
3 19:48:13 CREATE TABLE credit_cards (id VARCHAR(20) PRIMARY KEY, user_id INT, iban VARCHAR(50), pa... 0 row(s) affected
4 19:49:03 CREATE TABLE transactions (id VARCHAR(100) PRIMARY KEY, card_id VARCHAR(20), business_id V... 0 row(s) affected

Diagrama de relaciones



- 8.5 Transferencia de datos a las tablas definitivas

```
-- Companies
INSERT INTO companies
SELECT * FROM tmp_companies;
```

```
-- Users (americanos + europeos juntos)
INSERT INTO users (source_user_id, name, surname, phone, email,
birth_date, country, city, postal_code, address, signup_date,
user_segment, income_band, continent)
SELECT id, name, surname, phone, email, STR_TO_DATE(birth_date, '%b
%d, %Y'), country, city, postal_code, address, signup_date,
user_segment, income_band, 'America'
FROM tmp_american_users
UNION ALL
SELECT id, name, surname, phone, email, STR_TO_DATE(birth_date, '%b
%d, %Y'), country, city, postal_code, address, signup_date,
user_segment, income_band, 'Europe'
FROM tmp_european_users;

-- Credit cards
INSERT INTO credit_cards (id, user_id, iban, pan, pin, cvv, track1,
track2, expiring_date, card_type, card_renewal_flag)
SELECT id, user_id, iban, pan, pin, cvv, track1, track2,
STR_TO_DATE(expiring_date, '%m/%d/%y'), card_type, card_renewal_flag
FROM tmp_credit_cards;

-- Transactions
INSERT INTO transactions
SELECT id, card_id, business_id, timestamp, amount, declined,
product_ids, user_id, lat, longitude, discount_amount, tax_amount,
shipping_amount, channel, campaign_id, device_type, is_international,
decline_reason, distance_km
FROM tmp_transactions;

-- Error en decimales
ALTER TABLE transactions
MODIFY COLUMN lat DOUBLE,
MODIFY COLUMN longitude DOUBLE;
```

Se creó la tabla dim_table, para conectar o estructurar bien el diagrama estrella. Además, para cuando se realicen consultas de tiempo, no nos den error.

```sql
-- Crear la tabla de date
CREATE TABLE dim_date (
    date_key INT PRIMARY KEY,
    full_date DATE,
    day INT,
    month INT,
    month_name VARCHAR(20),
    quarter INT,
    year INT,
    weekday_number INT,
    weekday_name VARCHAR(20),
    is_weekend TINYINT
);

INSERT INTO dim date
```

Output

Action Output

#	Time	Action	Message
1	20:00:36	CREATE TABLE dim_date (date_key INT PRIMARY KEY, full_date DATE, day INT, month INT, mo...	0 row(s) affected

```sql
INSERT INTO dim_date
SELECT
    DATE_FORMAT(full_date, '%Y%m%d') AS date_key,
    full_date,
    DAY(full_date) AS day,
    MONTH(full_date) AS month,
    MONTHNAME(full_date) AS month_name,
    QUARTER(full_date) AS quarter,
    YEAR(full_date) AS year,
    WEEKDAY(full_date) AS weekday_number,
    DAYNAME(full_date) AS weekday_name,
    IF(WEEKDAY(full_date) >= 5, 1, 0) AS is_weekend
FROM (
    SELECT ADDDATE('2010-01-01', t) AS full_date
    FROM (
        SELECT ones.n + tens.n * 10 + hundreds.n * 100 + thousands.n * 1000 AS t
        FROM
            (SELECT 0 AS n UNION SELECT 1 UNION SELECT 2 UNION SELECT 3 UNION SELECT 4 UNION SELECT 5 UNION SELECT 6 UNION SELECT 7 UNION SE
            (SELECT 0 AS n UNION SELECT 1 UNION SELECT 2 UNION SELECT 3 UNION SELECT 4 UNION SELECT 5 UNION SELECT 6 UNION SELECT 7 UNION SE
            (SELECT 0 AS n UNION SELECT 1 UNION SELECT 2 UNION SELECT 3 UNION SELECT 4 UNION SELECT 5 UNION SELECT 6 UNION SELECT 7 UNION SE
            (SELECT 0 AS n UNION SELECT 1 UNION SELECT 2 UNION SELECT 3 UNION SELECT 4 UNION SELECT 5 UNION SELECT 6 UNION SELECT 7 UNION SE
    ) numbers
```

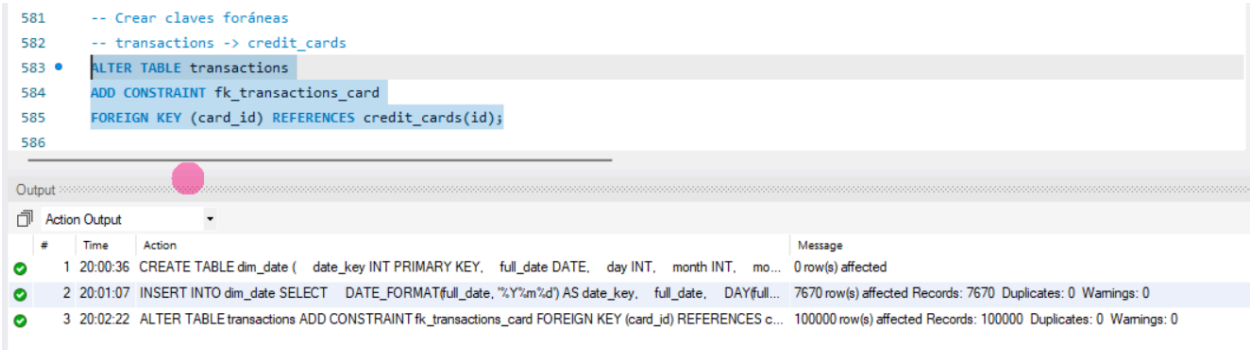
Output

Action Output

#	Time	Action	Message
1	20:00:36	CREATE TABLE dim_date (date_key INT PRIMARY KEY, full_date DATE, day INT, month INT, mo...	0 row(s) affected
2	20:01:07	INSERT INTO dim_date SELECT DATE_FORMAT(full_date, '%Y%m%d') AS date_key, full_date, DAY(full...	7670 row(s) affected Records: 7670 Duplicates: 0 Warnings: 0

-- 8.7 Crear claves foráneas para garantizar su integridad de la referencia de los datos

-- Creamos un índice porque me daba error al referenciar user_id en transaction, además para ahorro de tiempo al cargar datos.

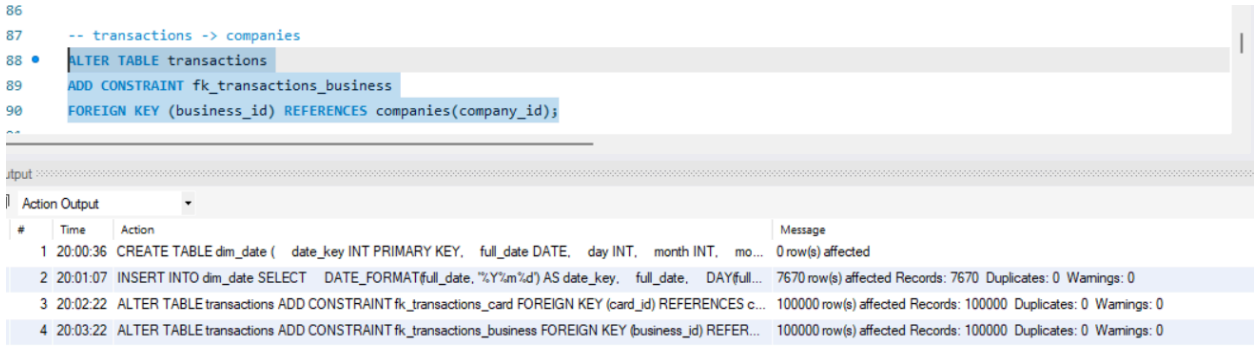


```
581     -- Crear claves foráneas
582     -- transactions -> credit_cards
583     ALTER TABLE transactions
584     ADD CONSTRAINT fk_transactions_card
585     FOREIGN KEY (card_id) REFERENCES credit_cards(id);
586
```

Output

Action Output

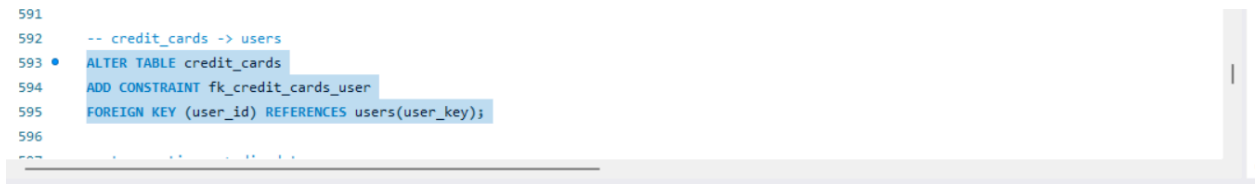
#	Time	Action	Message
1	20:00:36	CREATE TABLE dim_date (date_key INT PRIMARY KEY, full_date DATE, day INT, month INT, mo...	0 row(s) affected
2	20:01:07	INSERT INTO dim_date SELECT DATE_FORMAT(full_date, '%Y%m%d') AS date_key, full_date, DAY(full...	7670 row(s) affected Records: 7670 Duplicates: 0 Warnings: 0
3	20:02:22	ALTER TABLE transactions ADD CONSTRAINT fk_transactions_card FOREIGN KEY (card_id) REFERENCES c...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0



```
86
87      -- transactions -> companies
88      ALTER TABLE transactions
89      ADD CONSTRAINT fk_transactions_business
90      FOREIGN KEY (business_id) REFERENCES companies(company_id);
```

Action Output

#	Time	Action	Message
1	20:00:36	CREATE TABLE dim_date (date_key INT PRIMARY KEY, full_date DATE, day INT, month INT, mo...	0 row(s) affected
2	20:01:07	INSERT INTO dim_date SELECT DATE_FORMAT(full_date, '%Y%m%d') AS date_key, full_date, DAY(full...	7670 row(s) affected Records: 7670 Duplicates: 0 Warnings: 0
3	20:02:22	ALTER TABLE transactions ADD CONSTRAINT fk_transactions_card FOREIGN KEY (card_id) REFERENCES c...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0
4	20:03:22	ALTER TABLE transactions ADD CONSTRAINT fk_transactions_business FOREIGN KEY (business_id) REFER...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0



```
591
592     -- credit_cards -> users
593     ALTER TABLE credit_cards
594     ADD CONSTRAINT fk_credit_cards_user
595     FOREIGN KEY (user_id) REFERENCES users(user_key);
596
```

Output

Action Output

#	Time	Action	Message
1	20:00:36	CREATE TABLE dim_date (date_key INT PRIMARY KEY, full_date DATE, day INT, month INT, mo...	0 row(s) affected
2	20:01:07	INSERT INTO dim_date SELECT DATE_FORMAT(full_date, '%Y%m%d') AS date_key, full_date, DAY(full...	7670 row(s) affected Records: 7670 Duplicates: 0 Warnings: 0
3	20:02:22	ALTER TABLE transactions ADD CONSTRAINT fk_transactions_card FOREIGN KEY (card_id) REFERENCES c...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0
4	20:03:22	ALTER TABLE transactions ADD CONSTRAINT fk_transactions_business FOREIGN KEY (business_id) REFER...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0
5	20:04:17	ALTER TABLE transactions ADD CONSTRAINT fk_transactions_business FOREIGN KEY (business_id) REFER...	Error Code: 1826. Duplicate foreign key constraint name 'fk_transactions_business'
6	20:05:18	ALTER TABLE credit_cards ADD CONSTRAINT fk_credit_cards_user FOREIGN KEY (user_id) REFERENCES u...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0

```
608 •  ALTER TABLE transactions
609    ADD CONSTRAINT fk_transactions_date
610    FOREIGN KEY (date_key) REFERENCES dim_date(date_key);
611
612 •  SELECT * FROM information_schema.KEY_COLUMN_USAGE
613    WHERE TABLE_NAME = 'transactions'
614    AND TABLE_SCHEMA = 'operations';
615
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CONSTRAINT_CATALOG	CONSTRAINT_SCHEMA	CONSTRAINT_NAME	TABLE_CATALOG	TABLE_SCHEMA	TABLE_NAME	COLUMN_NAME	ORDINAL_POSITION	POSITION_IN_UN
def	operations	PRIMARY	def	operations	transactions	id	1	NULL
def	operations	fk_transactions_business	def	operations	transactions	business_id	1	1
def	operations	fk_transactions_card	def	operations	transactions	card_id	1	1
def	operations	fk_transactions_date	def	operations	transactions	date_key	1	1

KEY_COLUMN_USAGE 24 ×

Output

Action Output

#	Time	Action	Message
1	20:09:01	ALTER TABLE transactions ADD CONSTRAINT fk_transactions_date FOREIGN KEY (date_key) REFERENCE...	Error Code: 2013. Lost connection to MySQL server during query
2	20:09:48	ALTER TABLE transactions ADD CONSTRAINT fk_transactions_date FOREIGN KEY (date_key) REFERENCE...	Error Code: 1826. Duplicate foreign key constraint name 'fk_transactions_date'
3	20:09:51	ALTER TABLE transactions ADD CONSTRAINT fk_transactions_date FOREIGN KEY (date_key) REFERENCE...	Error Code: 1826. Duplicate foreign key constraint name 'fk_transactions_date'
4	20:10:15	SELECT * FROM information_schema.KEY_COLUMN_USAGE WHERE TABLE_NAME = 'transactions' AND T...	4 row(s) returned

Nos aseguramos de que existan las FK

```
643    -- ver las calves foráneas
644 •  SELECT CONSTRAINT_NAME, COLUMN_NAME, REFERENCED_TABLE_NAME, REFERENCED_COLUMN_NAME
645    FROM information_schema.KEY_COLUMN_USAGE
646    WHERE TABLE_NAME = 'transactions'
647    AND TABLE_SCHEMA = 'operations'
648    AND REFERENCED_TABLE_NAME IS NOT NULL;
649
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CONSTRAINT_NAME	COLUMN_NAME	REFERENCED_TABLE_NAME	REFERENCED_COLUMN_NAME
fk_transactions_business	business_id	companies	company_id
fk_transactions_card	card_id	credit_cards	id
fk_transactions_date	date_key	dim_date	date_key
fk_transactions_user	user_id	users	user_key

KEY_COLUMN_USAGE 27 ×

Output

Action Output

#	Time	Action	Message
1	09:25:09	SHOW VARIABLES LIKE 'secure_file_priv'	1 row(s) returned
2	10:35:42	SELECT CONSTRAINT_NAME, COLUMN_NAME, REFERENCED_TABLE_NAME, REFERENCED_COLUMN...	4 row(s) returned

-- 8.8 Borrar tablas temporales

```
618 •  DROP TABLE tmp_companies;
619 •  DROP TABLE tmp_american_users;
620 •  DROP TABLE tmp_european_users;
621 •  DROP TABLE tmp_credit_cards;
622 •  DROP TABLE tmp_transactions;
623
624    -- El diagrama se encuentra en el PDF Consultes_Tasca_Sprint2_N1.pdf
625
```

Output

Action Output

#	Time	Action	Message
4	20:10:15	SELECT * FROM information_schema.KEY_COLUMN_USAGE WHERE TABLE_NAME = 'transactions' AND ...	4 row(s) returned
5	20:11:32	DROP TABLE tmp_companies	0 row(s) affected
6	20:11:35	DROP TABLE tmp_american_users	0 row(s) affected
7	20:11:38	DROP TABLE tmp_european_users	0 row(s) affected
8	20:11:41	DROP TABLE tmp_credit_cards	0 row(s) affected
9	20:11:46	DROP TABLE tmp_transactions	0 row(s) affected

```
632 •  SELECT CONSTRAINT_NAME, COLUMN_NAME, REFERENCED_TABLE_NAME, REFERENCED_COLUMN_NAME
633    FROM information_schema.KEY_COLUMN_USAGE
634    WHERE TABLE_NAME = 'transactions'
635    AND TABLE_SCHEMA = 'operations'
636    AND REFERENCED_TABLE_NAME IS NOT NULL;
637
638    -- El diagrama se encuentra en el PDF Consultes_Tasca_Sprint2_N1.pdf
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CONSTRAINT_NAME	COLUMN_NAME	REFERENCED_TABLE_NAME	REFERENCED_COLUMN_NAME
fk_transactions_business	business_id	companies	company_id
fk_transactions_card	card_id	credit_cards	id
fk_transactions_date	date_key	dim_date	date_key
fk_transactions_user	user_id	users	user_key

KEY_COLUMN_USAGE 27

Output

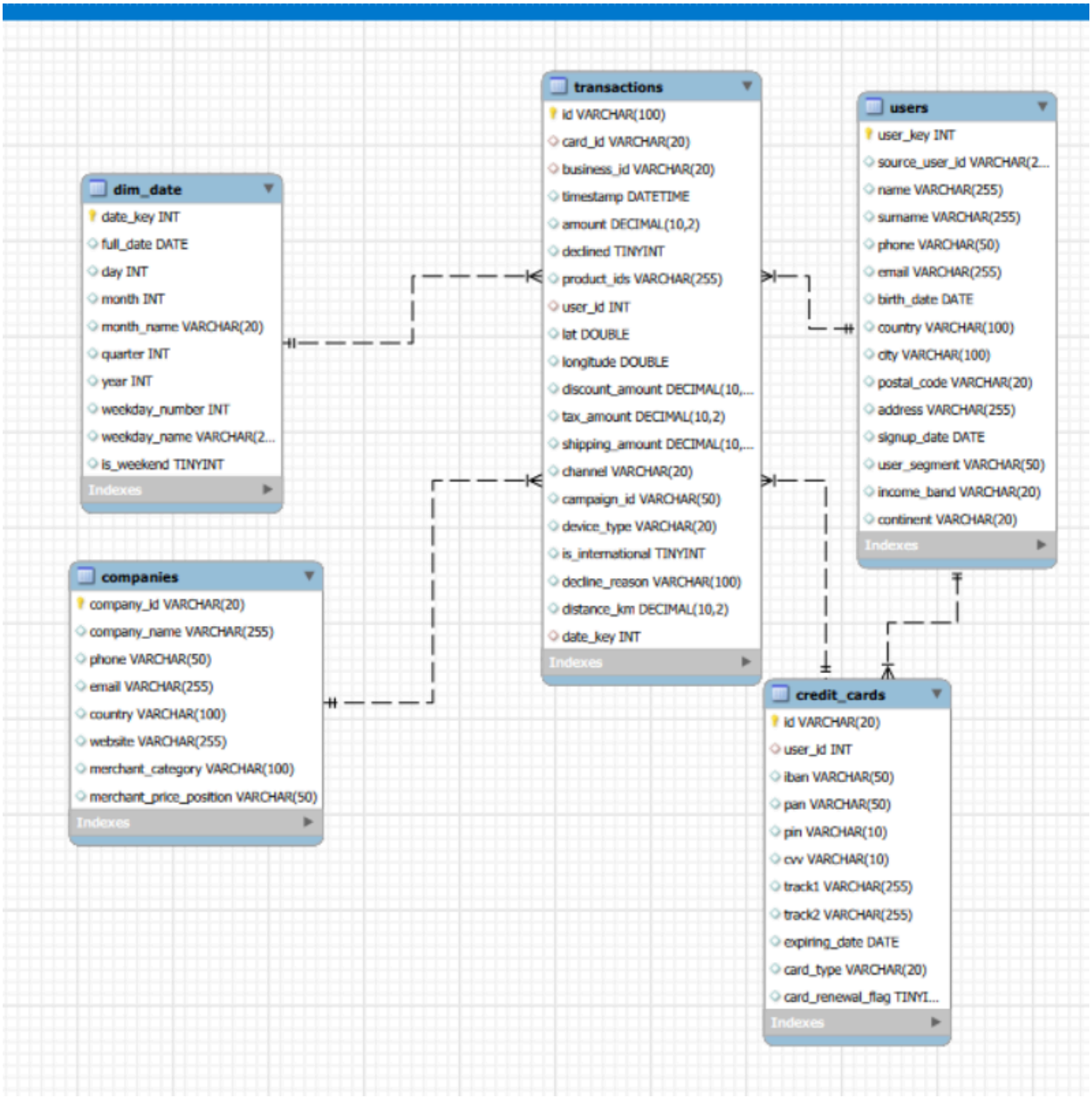
Action Output

#	Time	Action	Message
1	20:18:35	SELECT CONSTRAINT_NAME, COLUMN_NAME, REFERENCED_TABLE_NAME, REFERENCED_COLUMN...	4 row(s) returned

La tabla transaction es una tabla de hechos, que en un modelo estrella es la que contiene datos cuantitativos y es la que se relaciona con sus claves foráneas a las tablas padre en este caso llamadas dimensiones, las que contienen el contexto o la información.

También podemos visualizar en el diagrama el tipo de relaciones de 1:N, donde puede haber varias transacciones por ejemplo de un usuario o de una compañía.

Se agrega la tabla de tiempo para que haya una relación con este, la dimensión de tiempo es importante es por ello que se crea una tabla única para ello. Esto permite hacer análisis temporales de forma más eficiente y sin tener que calcular esas cosas en cada consulta.



/***Ejercicio 9**
Realiza una subconsulta que muestre a todos los usuarios con más de 80 transacciones utilizando al menos 2 tablas.*/

Nota: AHora usamos la BBDD operations a aqui y en los siguientes ejercicios

```
656
657 •  SELECT * FROM users u
658    WHERE EXISTS (
659    SELECT user_id
660    FROM transactions t
661    WHERE t.user_id = u.user_key
662    GROUP BY user_id
663    HAVING COUNT(user_id)>80);
664
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

user_key	source_user_id	name	surname	phone	email	birth_date	country	city	postal_code	address	
185	235	Medge	Nieves	044-518-5071	ac@icloud.couk	1993-06-11	Canada	Montreal	H1A 0A1	561-787 Pellentesque Street	2
289	855	Ulrwry	Mbrwmyxg	+57-915-8737	ulrwry.mbrwmyxg@example.com	1972-07-13	Canada	Vancouver	V5K 0A1	386 Mbrwmyxg St	2
318	1015	Ydxfjl	Yhswlpqn	+84-730-3250	ydxfjl.yhswlpqn@example.com	2000-11-13	Canada	Hamilton	L8E 0A1	209 Yhswlpqn St	2
454	1780	Tklciq	Cyhzykgv	+82-174-3802	tklciq.cyhzykgv@example.com	1955-02-13	Canada	Winnipeg	R2C 0A1	271 Cyhzykgv St	2

users 30 ×

Output

Action Output

#	Time	Action	Message
1	10:45:14	SELECT * FROM users u WHERE EXISTS (SELECT user_id FROM transactions t WHERE t.user_id = u.user_k...	4 row(s) returned

/***Ejercicio 10**

Muestra la media de amount por IBAN de las tarjetas de crédito en la compañía Donec Ltd.,
utiliza por lo menos 2 tablas.*/

```
672 •  USE operations;
673
674 •  SELECT cd.iban, c.company_name, AVG(t.amount) AS media_cantidad
675    FROM transactions t
676    JOIN credit_cards cd
677    ON t.card_id = cd.id
678    JOIN companies c
679    ON t.business_id = c.company_id
680    WHERE c.company_name = 'Donec Ltd'
681    GROUP BY cd.iban;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

iban	company_name	media_cantidad
CA90135577358677162610899O	Donec Ltd	361.010000
US72921264281695565142879 1	Donec Ltd	142.560000
US89945080323093327681925 1	Donec Ltd	246.900000
US58109743266832554114229 7	Donec Ltd	154.010000
CA44122626710585747357057 4	Donec Ltd	250.510000

Result 31

Output

Action Output

#	Time	Action	Message
1	10:52:20	SELECT cd.iban, c.company_name, AVG(t.amount) AS media_cantidad FROM transactions t JOIN credit_cards ...	371 row(s) returned

NIVEL II

/*__Ejercicio 01__
Identifica los cinco días que se generará la mayor cantidad de ingresos en la empresa por ventas.
Muestra la fecha de cada transacción junto con el total de ventas.*/

```
692 •  SELECT DATE(timestamp) AS fecha,
693    SUM(amount) AS Total
694    FROM transactions
695    GROUP BY DATE(timestamp)
696    ORDER BY Total DESC
697    LIMIT 5;
698
699
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

fecha	Total
2019-11-18	15840.24
2023-02-20	15728.40
2022-12-13	15129.20
2019-03-18	14896.47
2017-12-20	14367.96

Result 32

Output

Action Output

#	Time	Action	Message
1	10:55:29	SELECT DATE(timestamp) AS fecha, SUM(amount) AS Total FROM transactions GROUP BY DATE(timestamp)...	5 row(s) returned

/***Ejercicio 2**
Presenta el nombre, teléfono, país, fecha y amount, de aquellas empresas que
realizaron transacciones con un valor comprendido entre 350 y 400 euros y
en alguna de estas fechas: 29 de abril de 2015, 20 de julio de 2018 y 13 de marzo de 2024.
Ordena los resultados de mayor a menor cantidad.*/

```
710  SELECT company_name, phone, country, amount, DATE(timestamp)
711  FROM
712  JOIN companies c
713  ON t.business_id = c.company_id
714  WHERE amount BETWEEN 350 AND 400
715  AND (
716      DATE(timestamp) = '2015-04-29'
717      OR DATE(timestamp) = '2018-07-20'
718      OR DATE(timestamp) = '2024-03-13'
719  )
720  ORDER BY amount DESC;
721
```

Execute the statement under the keyboard cursor

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

company_name	phone	country	amount	DATE(timestamp)
Fringilla LLC	+1 150 784 1716	New Zealand	390.33	2015-04-29
Auctor Mauris Vel LLP	+32 133 447 8239	United States	383.58	2024-03-13
Mauris Institute	+75 473 619 9472	Sweden	369.20	2015-04-29
Pede Cum Ltd	+26 625 798 6126	Norway	358.64	2018-07-20
Netus Et Malesuada Ltd	+52 818 402 1638	Netherlands	352.21	2024-03-13

Result 38

Output

Action Output

#	Time	Action	Message
1	11:03:36	SELECT company_name, phone, country, amount, DATE(timestamp) FROM transactions t JOIN companies c O...	6 row(s) returned

/***Ejercicio 3**
Necesitamos optimizar la asignación de los recursos y
dependerá de la capacidad operativa que se requiera,
por lo que te piden la información sobre la cantidad de transacciones que realizan las empresas,
pero el departamento de recursos humanos es exigente y quiere un listado de las empresas
en las que especifiques si tienen igual o más de 400 transacciones o menos.*/

```
734
735 •  SELECT c.company_name, COUNT(*) AS cantidad_transacciones,
736        CASE
737        WHEN COUNT(*)>= 400 THEN 'Igual o más de 400 transacciones'
738        ELSE 'Cantidad de transacciones menores'
739        END AS capacidad_operativa
740    FROM transactions t
741    JOIN companies c
742    ON t.business_id = c.company_id
743    GROUP BY c.company_name;
744
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

company_name	cantidad_transacciones	capacidad_operativa
Ac Fermentum Incorporated	2401	Igual o más de 400 transacciones
Magna A Neque Industries	410	Igual o más de 400 transacciones
Fusce Corp.	447	Igual o más de 400 transacciones
Convallis In Incorporated	1514	Igual o más de 400 transacciones
Ante Iaculis Nec Foundation	472	Igual o más de 400 transacciones

Result 40

Output

Action Output

#	Time	Action	Message
1	11:07:43	SELECT c.company_name, COUNT(*) AS cantidad_transacciones, CASE WHEN COUNT(*)>= 400 THEN 'Igual...	100 row(s) returned

```
/*=============================================================================
===
Exercici 4
Elimina de la tabla transacción el registro con ID
000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de datos.
=============================================================================
=====================================*/
```

Verificamos si existe el usuario

```
/*=============================================================================
Exercici 4
Elimina de la taula transaction el registre amb ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base d
=============================================================================
USE operations;

SELECT id
FROM transactions
WHERE  id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id
000447FE-B650-4DCF-85DE-C7ED0EE1CAAD
NULL

transactions 42

Output

Action Output

#	Time	Action	Message
1	11:11:37	SELECT id FROM transactions WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	1 row(s) returned

Borramos

```
-- 4.2 Borramos
DELETE FROM transactions
WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
```

Action Output

#	Time	Action	Message
1	11:11:37	SELECT id FROM transactions WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	1 row(s) returned
2	11:14:31	DELETE FROM transactions WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	1 row(s) affected

```
/*================================================================================
============
Ejercicio 5
La sección de marketing desea tener acceso a información específica para realizar análisis
y estrategias efectivas. Se ha solicitado crear una vista que proporcione detalles clave sobre las compañías y sus transacciones.
 Será necesaria que crees una vista llamada VistaMarketing que contenga la siguiente información:
 Nombre de la compañía. Teléfono de contacto. País de residencia. Media de compra realizado por cada compañía.
 Presenta la vista creada, ordenando los datos de mayor a menor promedio de compra

================================================================================
==========================================*/
```

```sql
CREATE VIEW VistaMarketing AS
SELECT company_name, phone, country, AVG(t.amount) AS media_compra
FROM transactions t
JOIN companies c
ON T.business_id = c.company_id
GROUP BY company_name
ORDER BY  media_compra DESC;
```

Action Output

#	Time	Action	Message
1	11:25:08	CREATE VIEW VistaMarketing AS SELECT company_name, phone, country, AVG(t.amount) AS media_compra F...	0 row(s) affected

```sql
SELECT * FROM VistaMarketing;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

company_name	phone	country	media_compra
Ac Fermentum Incorporated	+64 601 244 6320	Germany	330.769338
Ut Semper Foundation	+10 283 500 6449	Sweden	307.372945
Orci Adipiscing Limited	+60 897 390 5820	United Kingdom	306.265648
Dolor Vitae Limited	+6 421 567 9195	France	305.952745
Nunc In Foundation	+13 267 741 2619	Italy	305.396141

VistaMarketing 44

Output

Action Output

#	Time	Action	Message
1	11:28:03	SELECT * FROM VistaMarketing	100 row(s) returned